

Test Project

Module B :: Back-end Development

Technical Skill Assessment (TSA)

12th May 2026

Web Technologies

Contents

 OhoShop (Online Shopping Mall).....	3
1. Project Overview	3
2. Tech Stack & Constraints	3
3. Detailed Requirements	3
4. Instructions to the Competitor	8
5. Other	9

OhoShop (Online Shopping Mall)

1. Project Overview

In the rapidly growing e-commerce market, the startup **OhoShop** is launching its own shopping mall platform that allows administrators to manage products and orders directly. Administrators can register and manage products and process orders, while general visitors can browse and purchase products. In this project, competitors are responsible for everything from database design to core feature implementation. This is an MVP, so focus on implementing the essential functionality. **Design is not a graded criterion.**

2. Tech Stack & Constraints

- **Tech Stack:** There are no restrictions on programming language or framework. You may freely choose any server-side rendering (SSR)-capable framework such as MERN, MEAN, Laravel, Python, Ruby On Rails.
- **Desktop Environment:** Responsive design is not required. The application only needs to work in a desktop Chrome browser.
- **Image Upload:** Product images must be JPG or PNG format only, with a file size limit of **5MB**. This validation must be performed **server-side**.
- **Database:** Categories must be managed in a separate table (`categories`). The products table (`products`) must reference `category_id` as a foreign key (FK).
- **Initial Data:** The following data must be created automatically on first run (via Seeder or Migration).
 - Admin account: Name Admin, Email `admin@OhoShop.local`, Password password (password must be stored as a hash)
 - At least 3 initial categories (e.g., Clothing, Electronics, Food)

3. Detailed Requirements

1. Authentication

1.1 Login and Session Management

The login page is accessible at `/admin/login`. The user enters their email and password. On successful login, the user is redirected to the admin dashboard. On failure, display the message **"Incorrect email or password."**

Accessing any route under `/admin` without being logged in redirects the user to the login page. Logging out destroys the session and redirects to the login page.

The currently logged-in admin's name must be displayed in the navigation on all admin pages.

2. Admin Features

2.1 Dashboard

The dashboard is the default admin page (/admin or /admin/dashboard) and is the first page shown after login.

Display the following 4 statistics as cards on the dashboard.

Card Label	Content
Total Products	Total number of products
Active Products	Number of products with status Active
Total Orders	Total number of orders
Total Revenue	Sum of all final payment amounts (after coupon discounts)

Also display a list of the 5 most recently placed orders. Each row must include: Order # , Customer, Total, Status, and Date.

2.2 Product Management

The product management page (/admin/products) displays all products in a table.

Table columns: **Name**, **Category**, **Price**, **Stock**, **Status**, **Created At**, Edit / Delete buttons.

Provide a search field to filter products by name. Submitting a search term displays only products whose name contains that keyword.

The **product registration form** (/admin/products/new) and **product edit form** (/admin/products/{id}/edit) must include the following fields.

- **Name** (required, max 100 characters)
- **Category** (required, dropdown — populated from categories in the database)
- **Price** (required, integer, 0 or greater)
- **Stock** (required, integer, 0 or greater)
- **Description** (<textarea>, optional)
- **Image** (file upload, JPG/PNG, max 5MB, **optional** — products can be registered without an image)
- **Status** (dropdown: Active / Out of Stock / Hidden)

When entering the edit form, existing data must be pre-filled in each field. If no new image is uploaded, the existing image is retained. If a new image is uploaded, delete the existing image file from the server and replace it with the new one.

When deleting a product, display the confirmation message **“Are you sure you want to delete this product?”** On confirmation, if the product appears in any order (order_items), reject the deletion and display **“Cannot delete a product with order history.”** If the product has no order history, delete it from the database and remove its image file from the server.

2.3 Category Management

The category management page (/admin/categories) allows adding, editing, and deleting categories.

- **Add:** Enter a category name and save. If the name already exists, display **“Category name already exists.”**
- **Edit:** Change the category name. Since products reference the category via FK, updating the category name in the categories table is automatically reflected in associated products.
- **Delete:** If any product references this category, reject the deletion and display **“Cannot delete a category that has products assigned.”**

2.4 Coupon Management

The coupon management page (/admin/coupons) allows registering and managing discount coupons.

The **coupon registration form** includes the following fields.

- **Coupon Code** (required, uppercase letters or digits only, 6–12 characters, must be unique)
- **Discount Type** (dropdown: Fixed / Percent)
- **Discount Value** (required, integer, 1 or greater. If Percent, must be between 1 and 100)
- **Minimum Order Amount** (required, integer, 0 or greater. Coupon cannot be applied to orders below this amount)
- **Expires At** (required, <input type="date">. Only future dates may be selected)
- **Usage Limit** (required, integer, 1 or greater)

Coupon list table columns: **Code, Type, Value, Min. Order, Expires At, Remaining Uses**, Delete button. Remaining uses are managed via the remaining_uses column, initialized to the Usage Limit value on registration and decremented by 1 each time the coupon is used.

2.5 Order Management

The order management page (/admin/orders) displays all orders in a table.

Table columns: **Order #, Customer, Phone, Total, Status, Date.**

Provide a dropdown to filter orders by status. Status values are Pending / Processing / Shipped / Delivered / Cancelled. Only orders matching the selected status are shown.

The order detail page (/admin/orders/{id}) must display the following information.

- Customer Info: Name, Phone, Shipping Address
- Order Items: Product (product name snapshot), Unit Price, Qty, Subtotal
- Coupon: Applied coupon code and discount amount. Display “None” if no coupon was applied.
- Total: Final payment amount
- **Order Status** dropdown (Pending / Processing / Shipped / Delivered / Cancelled) and a Save button. On successful save, display “Order status updated.” on the same page.
- When the status is changed to **Cancelled**, restore the stock of all products in the order by their ordered quantities. If the order is already Cancelled and the status is set to Cancelled again, do not restore stock a second time.

3. Public Pages

3.1 Home — Product List

The home page (/) displays products with status **Active** as cards. Each card shows the product image (display a grey placeholder area if no image exists), product name, category name, and price.

Display **12 items per page** with pagination. Sort by most recently registered first. Provide category filter links. Clicking a category shows only **Active** products in that category. Pagination must retain the current category filter.

Provide a keyword search field. Products are searched by name. The category filter and keyword search must be applied simultaneously. Pagination must retain the search conditions.

3.2 Product Detail

The product detail page is accessible at /products/{id}. Return a 404 page for non-existent IDs or products with status **Hidden**.

The product detail page displays the product image (grey placeholder if no image), product name, category name, price, stock quantity, and description.

Products with stock of 0 or status **Out of Stock** show an “Out of Stock” message and have the purchase button disabled.

Products with stock of 1 or more and status **Active** show a quantity input (<input type="number">, min 1, max current stock) and a “Buy Now” button. Clicking the button passes the product ID and quantity as query strings and navigates to the checkout page (/checkout?product_id={id}&quantity={n}).

3.3 Checkout

The checkout page (/checkout) reads `product_id` and `quantity` from the query string and displays the order summary (product name, unit price, quantity, subtotal). The server must validate the following conditions and redirect to the home page (/) if any are not met.

- The product with `product_id` does not exist
- The product status is **Hidden** or **Out of Stock**
- Stock is 0
- `quantity` is less than 1 or exceeds current stock

The page includes a form for the buyer's Name, Phone, and Shipping Address. All fields are required and orders are only processed after passing **server-side validation**.

There is a **Coupon Code** input field and an **"Apply"** button. When **"Apply"** is clicked, use `fetch` to send a POST request to the `/checkout/coupon` endpoint in the format below, and reflect the result on the page without a full reload. This endpoint must be implemented by the competitor.

- Request Body (JSON): { "coupon_code": "ABCD1234", "subtotal": 30000 }
- Success Response (JSON): { "valid": true, "discount_type": "fixed"|"percent", "discount_value": 3000, "discount_amount": 3000, "final_amount": 27000 }
- Failure Response (JSON): { "valid": false, "message": "error message" }
- Valid coupon: Calculate the discount and display the updated Subtotal, Discount, and Total on the page.
 - Fixed discount: Subtract the discount value directly from the subtotal.
 - Percent discount: Deduct $\text{subtotal} \times \text{discount rate} / 100$. Round down (floor) any decimal.
 - The final amount must not go below 0.
- Invalid coupon: Display the appropriate error message for each case.
 - Code does not exist: **"Invalid coupon code."**
 - Coupon has expired: **"This coupon has expired."**
 - Usage limit reached: **"This coupon has reached its usage limit."**
 - Below minimum order amount: **"This coupon requires a minimum order of {n}."**

Process the following as a **single transaction** when submitting the order.

1. Save order information to the orders table (buyer name, phone, shipping address, applied coupon ID — NULL if none, discount amount — 0 if none, final payment amount).
2. Save order item information to the order_items table (order ID, product ID, **product name snapshot**, quantity, unit price). The product name must be saved as it was at the time of the order so that it displays correctly even if the product is later deleted or renamed.
3. Deduct the ordered quantity from the product's stock (stock).
4. If a coupon was applied, decrement the coupon's remaining_uses by 1.

If stock is insufficient for the requested quantity, roll back the transaction and display **"Not enough stock."**

On successful order submission, redirect to the order complete page (/orders/{id}/complete). This page is accessible without authentication and displays the Order #, order item details, Discount, and Total.

3.4 Layout

All public pages include a header and footer. The header contains the site name (**OhoShop**) and a link to the home page. Each page must have an appropriate <title> tag reflecting the page content.

Admin panel pages include a sidebar or top navigation. Navigation menu items: **Dashboard, Products, Categories, Coupons, Orders.**

4. Submission Guidelines

- Upload all deliverables to the server directory /xx_module_b/.
- All work will be evaluated on remote server strictly.

4. Instructions to the Competitor

- You may build the system in a sub-folder or on a different port if necessary.
- Adjust paths accordingly so that /XX_module_b/ routes correctly to your project.
- Store all data in a database.
- Provide:

- A database dump file with sample data
- ER diagram
- A readme.txt explaining how to run the project

5. Other

- If using Node.js, ensure correct platform-specific handling of node_modules
- Temporary or build folders must not be included in the submission
- The project will be assessed using **Google Chrome Web Browser**
- The implementation must follow this technical specification exactly
- Postman collection must be provided to verify the API end-points.